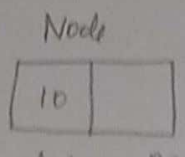
Linked List

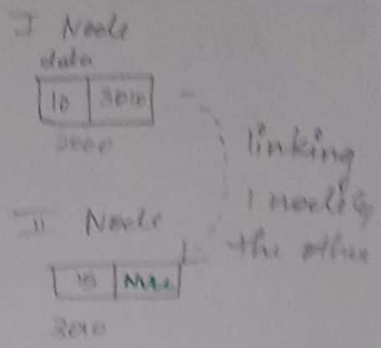
Linked list is a staggered Allocation as arrays are contigious memory Allocation.

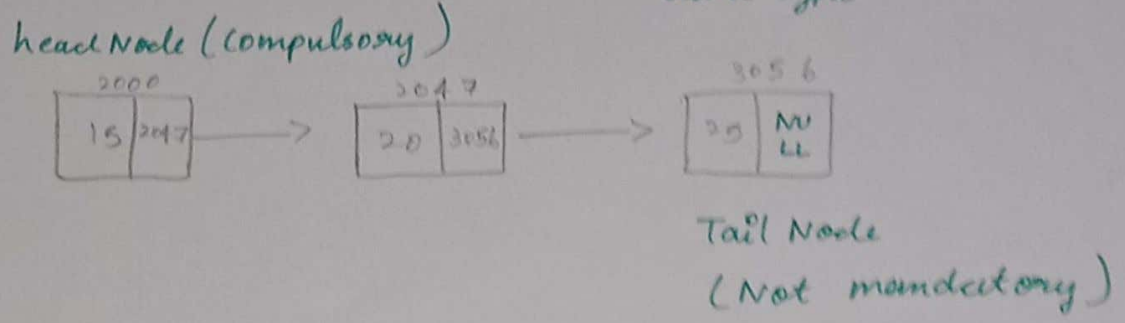


To create a Node

```
Class Node {
    int data;
    Node next;
}
```

Node → kind of data type





Types of LL:

1. singly LL 2. Doubly LL 3. circular LL

To create a head : Node head.

getting value in LL

```
function get (index)
Node ptr = head;
for (int i=0; i<index; i++)
    ptr = ptr.next
return ptr.data;
```

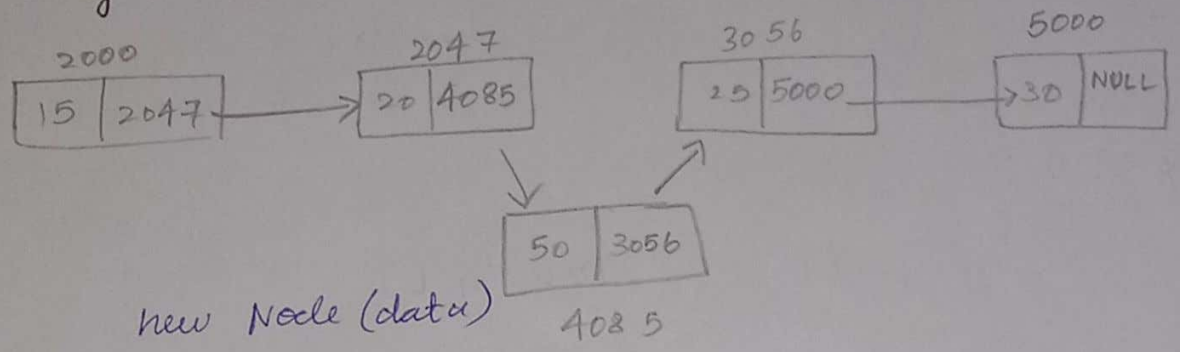
set value in LL.

```
function set (index, value)
{ Node ptr = head;
for (int i=0; i<index; i++)
    ptr = ptr.next
ptr.data = value; }
```

Search

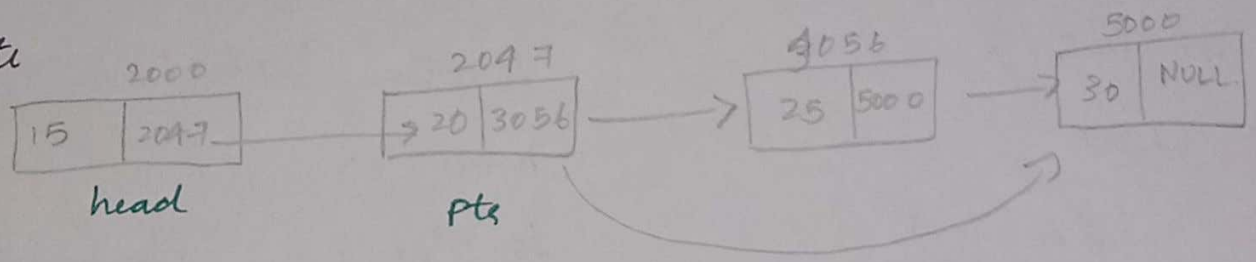
```
function search ( value )
{ count = 0
  Node ptr = head ;
  while (Ptr . next ! = null )
  {
      count ++ ;
      if (Ptr. data = = value) {
          Print ("found"); }
      Ptr = ptr . next      }.
```

inserting




```
new Node (data)
newNode . next = ptr . next ;
ptr . next = newNode.
```

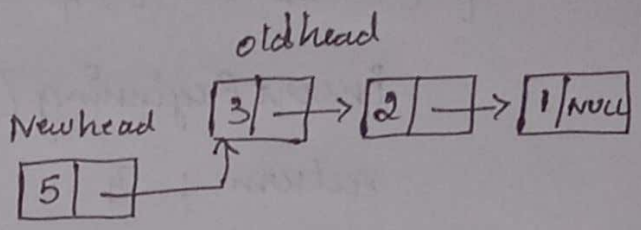
Delete

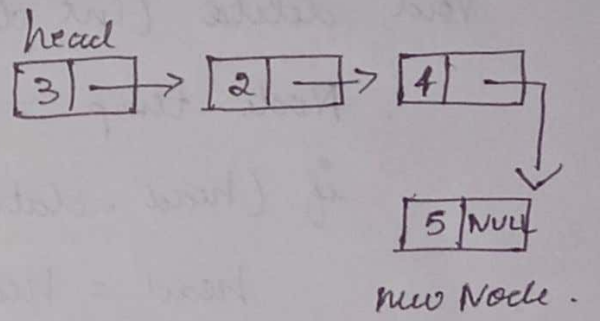



```
ptr . next = ptr . next . next ;
```

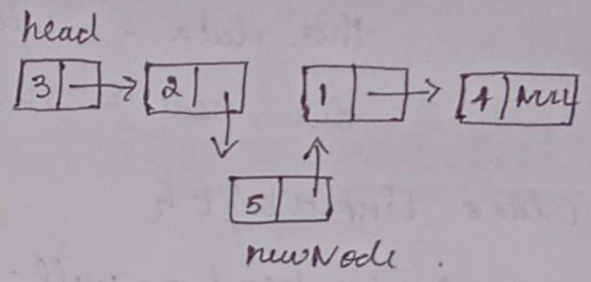
```
class Node {
    int data;
    Node next;
    Node (int data) {      // constructor
        this.data = data;
        this.next = null; }

class LinkedList {
    Node head = null;
    void insertBeginning (int data) {
        Node newNode = new Node (data);
        newNode.next = head;
        head = newNode; }
    void insertAtEnd (int data) {
        Node newNode = new Node (data);
        if (head == null) {
            head = newNode;
            return; }
        Node temp = head;
        while (temp.next != null) {
            temp = temp.next; }
        temp.next = newNode; }
```

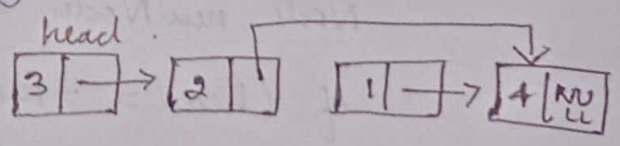




```
void insertIndex (int data, int index){
  if (index < 0) {
      System.out.println("Invalid
                          index");
      return ; }
  if (index == 0) {
      insertBeginning (data);
      return ; }
  Node newNode = new Node (data);
  Node temp = head ;
  for (int i=0 ; i<index -1 ; i++) {
      temp = temp.next ; }
  newNode.next = temp.next ;
  temp.next = newNode ; }
```



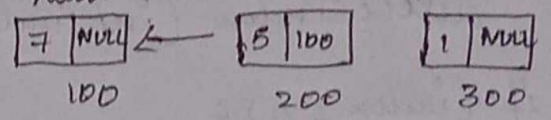

```
void delete (int data) {
  Node temp = head ;
  if (head.data == head) {
      head = head.data ;
      return ; }
  While (temp.next != null && temp.next.data != data)
  { temp = temp.next ; }
  if (temp.next == null ) {
      System.out.println("Invalid Index");
      return ; }
  temp.next = temp.next.next ;
}
```



```
void deleteIndex (int index) {
    if (index < 0) {
        System.out.println("Invalid
                                Index");
        return; }
    Node temp = head;
    if (index == 0) {
        head = head.next;
        return; }
    for (int i=0; i< index-1; i++) {
        temp = temp.next; }
    temp.next = temp.next.next; }

void search (int data) {
    Node temp = head;
    while (temp != null) {
        if (temp.data == data) {
            System.out.println("true");
            return; }
        temp = temp.next; }
    System.out.println("false");
}
```

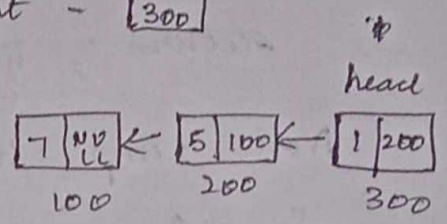
```
void reverse() {
    Node prev = null;
    Node curr = head;
    Node next = null;
    while (curr != null){
        next = curr.next;
        curr.next = prev; //reversing a pointer
        prev = curr;
        curr = next; }
    return prev ; }
```



prev - 100

curr - 200

next - 300



prev = 300

curr = NULL

next = NULL

```
void display() {
    System.out.println();
    Node temp = head;
    while (temp != null){
    System.out.println(temp.data + " --> ");
    temp = temp.next ; }
    System.out.println("Null"); }
public static void Main(String[] args) {
    LinkedList list = new LinkedList();
    list.insertAtBeginning(10);
    list.insertAtEnd(20);
    list.insertIndex(1,15);
    list.display(); // 10 -> 15 -> 20 -> NULL
```

```
list.reverse();
list.display(); // 20 -> 15 -> 10 -> NULL
list.delete(15)
list.display(); // 20 -> 10 -> NULL
list.deleteIndex(0);
list.display(); // 10
System.out.println("Search 20: " + list.search(20));
                                        // false
System.out.println("Search 10: " + list.search(10));
                                        // True.
}}
```

Time complexity of each functions.

1. insertBeginning - O(1)
2. insertAtEnd - O(n)
3. insertIndex - O(n)
4. delete - O(n)
5. reverse - O(n)
6. deleteIndex - O(n)
7. Search - O(n).
8. display - O(n).